

Computer Languages

Language

Means of communication

Arbitrary

Must be agreement between parties

Codified as grammar

Atom

Smallest particle or atom is a word

Subatomic particles exist: syllables, phonemes, and graphemes

We will restrict ourselves to words

Words

Word associates a concept

In natural language, we have types of words:

Nouns

Verbs

Adverbs

Adjectives

Pronouns

Prepositions

Conjunctions

Interjections

Types of Words

Nouns:

Name a person, place, thing, or idea (e.g., dog, city, happiness).

Pronouns:

Take the place of a noun to avoid repeating it (e.g., he, she, they, it).

Verbs:

Show an action or a state of being (e.g., run, is, eat).

Adjectives:

Describe or modify a noun or a pronoun (e.g., big, happy, red).

Adverbs:

Describe or modify a verb, an adjective, or another adverb, often telling how, when, or where (e.g., quickly, very, yesterday).

Prepositions:

Show the relationship of time, place, or position between words (e.g., in, on, under, before).

Conjunctions:

Connect words, phrases, or clauses together (e.g., and, but, because).

Interjections:

Express strong emotion or sudden feeling (e.g., wow, ouch, hey).

Combining Words

An ordered set of words creates a statement

With a set of words S , almost infinite (factorial) number of combinations.

Some combinations have meaning

Most don't

Combining Statements

An ordered set of statements creates a conversation or program

Extremely large set of possible conversations (programs).

Some conversations (programs) have meaning or usefulness

Most don't

Language Hierarchy

Program

Statement[s]

Word[s]

Syntax versus Semantics

Correct syntax follows the language rules

Correct semantics is the correct meaning

Correctness requires syntax and semantics

Context

The situation, setting, or environment

The surrounding that helps explain a word or statement

Context Free

The situation, setting, or environment does not alter the meaning of a word or statement

My **name** is Bill.

What is your **name**?

Context Sensitive

The situation, setting, or environment alters the meaning

Lance Armstrong wanted the Tour de France
record wins

Please press the **record** button

Meaning changes based the word environment

Keywords

Reserved word

Only has one use

Context free

Only has one meaning

Examples: if procedure module else while

Symbols

Abstraction

Creates an association

May change or mutate an association (mutable)

Function of time

Known as a variable

Example : $\alpha \leftarrow 1$ α is the symbol or variable

Reducing Complexity

Using keywords and allowing only one possible context (context free) makes the language simpler and easier to understand

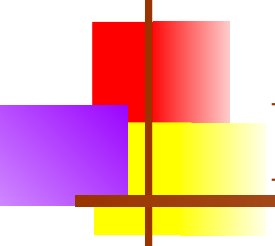
Most programming languages are almost context free

C, C++, and Rust are almost context free

Formal Language Specification

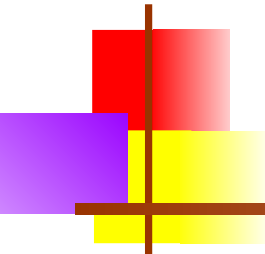
We can use another language to describe our language of interest

One such description is BNF



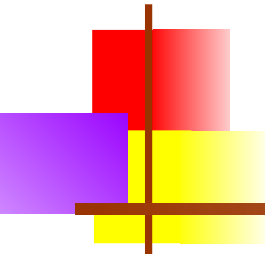
Metalanguages

- A *metalanguage* is a language used to talk about a language (usually a different one)
- We can use English as its own metalanguage (e.g. describing English grammar in English)
- It is essential to distinguish between the metalanguage terms and the object language terms



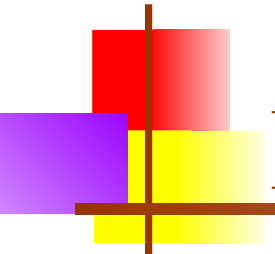
BNF

- BNF stands for either Backus-Naur Form or Backus Normal Form
- BNF is a metalanguage used to describe the grammar of a programming language
- BNF is formal and precise
 - BNF is a notation for **context-free grammars**
- BNF is essential in compiler construction
- There are many dialects of BNF in use, but...
- ...the differences are almost always minor



BNF

- $\langle \rangle$ indicate a *nonterminal* that needs to be further expanded, e.g. $\langle \text{variable} \rangle$
- Symbols not enclosed in $\langle \rangle$ are *terminals or keywords*; they represent themselves, e.g. *if*, *while*, *(*
- The symbol $::=$ means *is defined as*
- The symbol $|$ means *or*; it separates alternatives, e.g. $\langle \text{addop} \rangle ::= + \mid -$
- This is *all there is* to “plain” BNF; but we will discuss *extended* BNF (**EBNF**) later in this lecture



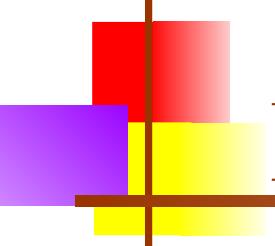
BNF uses recursion

- $\langle \text{integer} \rangle ::= \langle \text{digit} \rangle \mid \langle \text{integer} \rangle \langle \text{digit} \rangle$

or

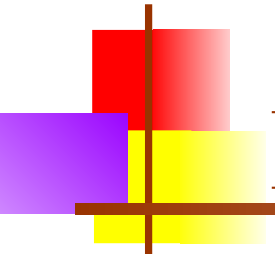
- $\langle \text{integer} \rangle ::= \langle \text{digit} \rangle \mid \langle \text{digit} \rangle \langle \text{integer} \rangle$

- Recursion is all that is needed (at least, in a formal sense)



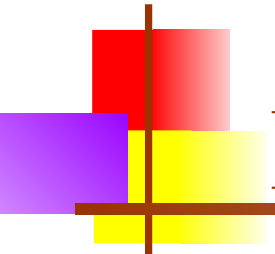
Recursion

- Recursion is the process where a concept, function, or process depends on or calls a simpler version of itself to solve a problem
- A recursive solution or definition requires two main parts to work properly without running forever
 - **Base case:** The simplest scenario or stopping condition that ends the recursion and returns a direct answer
 - **Recursive case:** The step where the process breaks the problem down and calls or applies itself to a smaller piece of the original problem.



Recursion

- $\langle \text{integer} \rangle ::= \langle \text{digit} \rangle$
 - Base case



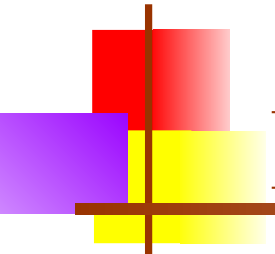
BNF uses recursion

- $\langle \text{integer} \rangle ::= \langle \text{integer} \rangle \langle \text{digit} \rangle$

or

$\langle \text{integer} \rangle ::= \langle \text{digit} \rangle \langle \text{integer} \rangle$

- Recursion case



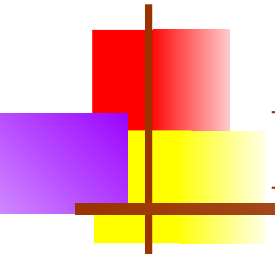
BNF uses recursion

- $\langle \text{integer} \rangle ::= \langle \text{digit} \rangle \mid \langle \text{integer} \rangle \langle \text{digit} \rangle$

or

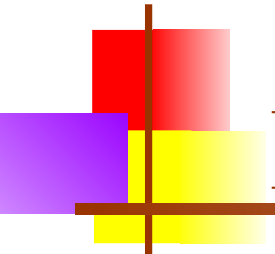
- $\langle \text{integer} \rangle ::= \langle \text{digit} \rangle \mid \langle \text{digit} \rangle \langle \text{integer} \rangle$

- Recursion is all that is needed (at least, in a formal sense)
- "Extended BNF" allows repetition as well as recursion
- Repetition is usually better when using BNF to construct a compiler



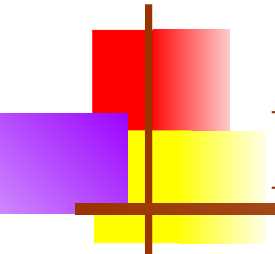
BNF Examples I

- `<digit> ::=`
 `0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`
- `<if statement> ::=`
 `if ( <condition> ) <statement>`
 `| if ( <condition> ) <statement>`
 `else <statement>`



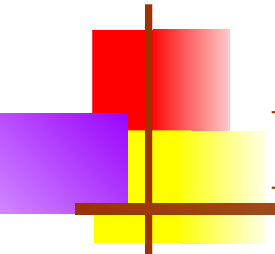
BNF Examples II

- $\langle \text{unsigned integer} \rangle ::=$
 $\langle \text{digit} \rangle \mid \langle \text{unsigned integer} \rangle \langle \text{digit} \rangle$
- $\langle \text{integer} \rangle ::=$
 $\langle \text{unsigned integer} \rangle$
 $\mid + \langle \text{unsigned integer} \rangle$
 $\mid - \langle \text{unsigned integer} \rangle$



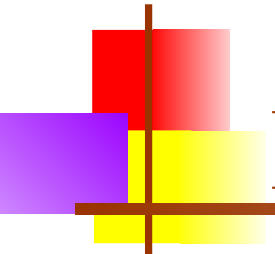
BNF Examples III

- $\langle \text{identifier} \rangle ::=$
 - $\langle \text{letter} \rangle$
 - | $\langle \text{identifier} \rangle \langle \text{letter} \rangle$
 - | $\langle \text{identifier} \rangle \langle \text{digit} \rangle$
- $\langle \text{block} \rangle ::= \{ \langle \text{statement list} \rangle \}$
- $\langle \text{statement list} \rangle ::=$
 - $\langle \text{statement} \rangle$
 - | $\langle \text{statement list} \rangle \langle \text{statement} \rangle$



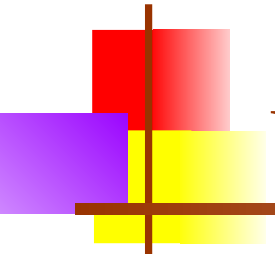
BNF Examples IV

- `<statement> ::=`
 - `<block>`
 - | `<assignment statement>`
 - | `<break statement>`
 - | `<continue statement>`
 - | `<do statement>`
 - | `<for loop>`
 - | `<goto statement>`
 - | `<if statement>`
 - | `...`



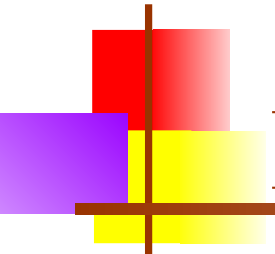
Extended BNF

- The following are pretty standard:
 - [] enclose an optional part of the rule
 - Example:
`<if statement> ::=`
`if ( <condition> ) <statement> [ else <statement> ]`
 - { } mean the enclosed can be repeated any number of times (including zero)
 - Example:
`<parameter list> ::= ( )`
`| ( { <parameter> , } <parameter> )`



Variations

- The preceding notation is the original and most common notation
 - BNF was designed before we had boldface, color, more than one font, etc.
 - A typical modern variation might:
 - Use boldface to indicate multi-character terminals
 - Quote single-character terminals (because boldface isn't so obvious in this case)
- Example:
 - *if_statement* ::=
if "(" *condition* ")" *statement* [**else statement**]



Limitations of BNF

- No easy way to impose length limitations, such as maximum length of variable names
- No easy way to describe ranges, such as 1 to 31
- No way *at all* to impose distributed requirements, such as, a variable must be declared before it is used
- Describes *only syntax, not semantics*
- Nothing clearly better has been devised

Classifying Computer Languages

Hardware (binary) level: assembly languages

High Level (text) level: human written languages
translated to binary code

Translation to Binary

Compiled language: all statements at once
C, C++, Rust

Interpreted language: a statement at a time
Python, Bash, Tcl

Types of Computer Languages

- **Procedural Programming**
- **Object-Oriented Programming (OOP)**
- **Functional Programming**
- **Scripting Languages**
- **Logic Programming**

Types of Computer Languages

Procedural Programming:

- Follows a strict, top-down sequence of statements or commands to achieve an output.
- Organized around modules known as functions or procedures.
- Examples: C, Pascal, and BASIC.

Types of Computer Languages

Object-Oriented Programming (OOP):

- Treats software programs as a collection of interacting "objects".
- Combines data fields (attributes) and behavior (methods) into singular structures.
- Examples: Java, Python, and C++

Types of Computer Languages

Functional Programming:

- Focuses on evaluating mathematical functions rather than executing a sequential list of steps.
- Prioritizes state immutability to ensure predictable program behavior.
- Examples: Haskell, Scala, and F#

Types of Computer Languages

Scripting Languages:

- Designed to automate specific tasks or manage dynamic actions within existing applications.
- Examples: Bash, Perl, and Tcl.

Types of Computer Languages

Logic Programming:

- Rather than giving step-by-step commands, the programmer inputs a framework of facts and rules.
- The computer's logic system figures out how to find a valid solution.
- Example: Prolog

Imperative vs Declarative

Imperative

- Explicit control of statement sequence
- Explicit control of states
- Explicit control of flow

- Burden on Programmer
- Can be efficient

Declarative

Declarative

- Focus on result rather than how to achieve it
- Implicit control of states
- Implicit control of statement flow

- Burden on the compiler
- Less efficient in general

Imperative Example

Python:

```
numbers = [1, 2, 3, 4, 5]
even_numbers = []
for num in numbers:
    if num % 2 == 0:
        even_numbers.append(num)
```

Declarative Example

Python:

```
numbers = [1, 2, 3, 4, 5]
```

```
even_numbers = [num for num in numbers if num % 2 == 0]
```

Declarative Example

SQL:

```
SELECT * FROM users WHERE age > 21;
```

Class study

C, C++, Rust:

Focus on imperative programming